

Real-Time Bangladeshi Taka Banknote Detection via YOLOv8 Trained on Synthetically Composited Real-World Backgrounds

[Author Name(s)]

Department of Computer Science and Engineering, Independent University, Bangladesh

[email(s)]@iub.edu.bd

Abstract

Automatic recognition of paper currency has practical value for assistive technology, automated cash handling, and retail self-service, yet publicly available Bangladeshi Taka (BDT) datasets consist almost exclusively of clean, close-up, full-frame note scans. A model trained on such images learns that “the entire frame is the note,” an assumption that collapses on a live camera feed, where a note appears at an arbitrary scale, angle, and position against cluttered real backgrounds. We reformulate BDT recognition as an object-detection problem and address the resulting synthetic-to-real domain gap with a copy-paste compositing pipeline: 5,073 background-free note images spanning all nine circulating denominations (2, 5, 10, 20, 50, 100, 200, 500, and 1000 Taka) are pasted onto 1,500 real photographs from the COCO dataset under randomised perspective warp, rotation, scale, photometric jitter, motion blur, sensor noise, and JPEG re-compression, with exact bounding-box labels computed analytically from the paste geometry, requiring zero manual annotation. The resulting 22,315-image dataset is used to fine-tune a COCO-pretrained YOLOv8s detector for 80 epochs. On a held-out test split of 2,238 synthetically composited images, the model achieves 0.995 mAP@0.5, 0.847 mAP@0.5:0.95, 0.998 precision, and 0.997 recall, at 7.3 ms per image (~135 FPS) on a laptop GPU, exceeding comparable published detection baselines that rely on manually annotated data. We report these results together with an explicit discussion of their principal limitation: evaluation to date is entirely on held-out synthetic composites rather than independently captured real-world photographs, and we treat closing that specific gap as the paper’s most important item of future work rather than a minor caveat.

Index Terms—object detection, YOLOv8, Bangladeshi currency, synthetic data, domain randomization, copy-paste augmentation, transfer learning, real-time inference

I. INTRODUCTION

Automatic recognition of paper currency has practical value in assistive technology for the visually impaired, automated cash-handling machines, retail self-service, and financial automation. For Bangladeshi Taka in particular, a useful system must not only classify which note is present but also detect and localise it within a cluttered scene in real time, so that it can operate from a phone or webcam held by a user in an uncontrolled environment.

Most existing BDT recognition work frames the task as whole-image classification on clean, centred note images. Such models perform well on their own curated test sets but degrade sharply in the field, because real camera frames contain background clutter, perspective distortion, motion blur, variable lighting, and notes that occupy only a fraction of the frame. This work targets that gap directly by treating the problem as object detection and by engineering the training data so that its statistics resemble real webcam imagery rather than studio scans. This reformulation was motivated by a concrete failure observed during early development: a first-pass pipeline that labelled the entire frame as the note scored well on its own synthetic test set yet failed to detect

notes reliably on a live webcam, which is what prompted the compositing-based redesign presented here.

Our contributions are: (1) a reproducible synthetic data-generation pipeline that converts a background-free classification dataset into a large, richly augmented detection dataset with automatically generated, pixel-perfect bounding-box labels, requiring no manual annotation; (2) a fine-tuned YOLOv8s detector that attains 0.995 mAP@0.5 on a held-out test set while running above 100 FPS on a laptop GPU; (3) a deployable real-time webcam application with automatic low-light correction and a TorchScript export for dependency-free inference; and (4) an explicit account of the evaluation’s principal limitation, held-out synthetic testing rather than independently captured real-world validation, reported alongside the results rather than omitted.

II. RELATED WORK

Early Bangladeshi banknote recognition relied on classical neural networks with hand-engineered features; a neural network using axis-symmetrical masks reported an average classification accuracy of 98.57% across eight denomi-

nations [1]. More recent approaches use deep convolutional networks and transfer learning: lightweight CNNs and pre-trained backbones have been applied to real-time classification for visually impaired users [2], and dual-stream MobileNet/EfficientNet architectures have been proposed for robust recognition [3].

For the detection formulation specifically, single-stage detectors of the YOLO family are the dominant choice because of their speed. A YOLOv5-based Taka detector trained on 3,111 hand-annotated images reported precision and recall of 0.95 and 0.93 [4], and a YOLOv8-nano model with a custom Squeeze-and-Excitation head for multi-currency (USD/EUR/BDT) detection reported mAP@0.5 of 0.972 and mAP@0.5:0.95 of 0.804 at 3.1 ms per image [5]. Our work differs in its explicit focus on closing the synthetic-to-real domain gap through domain-randomised copy-paste compositing, an approach shown to be effective for synthetic-to-real object detection generally [6]. Whereas the YOLOv5 and YOLOv8n-SE baselines above annotate real photographs by hand, we generate an order of magnitude more labelled data automatically, with pixel-perfect boxes and full control over the distribution of scale, angle, and background.

III. DATASET AND DATA ENGINEERING

A. Source Currency Dataset (Foreground)

The banknote imagery is drawn from the BanglaTaka dataset, “A Diverse Image Dataset for Bangladeshi Currency Recognition,” openly available on Mendeley Data [7]. It contains 5,073 background-free images of BDT paper currency spanning all nine circulating denominations, captured with a variety of mobile-phone cameras and pre-processed to give clean, cropped notes on a plain field (Table 1). Because each image is already a tight crop of a single note, it is ideal foreground material for compositing.

Table 1: Per-denomination image counts, source BanglaTaka dataset

Denom.	2	5	10	20	50	100	200	500	1000
Count	445	660	419	480	416	565	423	1243	422

Total: 5,073 images.

B. Background Dataset

Real-world backgrounds were sourced from the COCO val2017 split [8], a widely used collection of natural photographs of desks, hands, indoor rooms, streets, and outdoor scenes. We downloaded 1,500 images programmatically. Using genuine photographic backgrounds, rather than solid colours or synthetic noise as in an earlier internal iteration of this pipeline, was found to be the single most important factor in generalising to real camera frames, since it forces the detector to separate a note from realistic clutter, texture, and lighting.

C. The Domain-Gap Problem

A model trained only on background-free BanglaTaka images learns that a note fills the entire frame on a plain background. When such a model is shown a webcam frame, where the note is small, rotated, and surrounded by objects, its assumptions break and detection collapses. This was observed directly in an early version of this system, which scored highly on synthetic tests yet failed to detect notes on a live webcam. Compositing notes onto real backgrounds recreates deployment conditions at training time and eliminates this gap.

D. Copy-Paste Compositing Process

Each source note is pasted onto a randomly chosen, randomly cropped COCO background after a chain of randomised transformations (Fig. 1). Because the placement and scale of each pasted note are known exactly at generation time, the corresponding YOLO-format bounding-box label is computed directly from that placement:

$$x_c = (x + w/2)/S, \quad y_c = (y + h/2)/S \quad (1)$$

$$\text{box}_w = w/S, \quad \text{box}_h = h/S \quad (2)$$

where a note of pixel size (w, h) is pasted at top-left corner (x, y) on a canvas of size $S = 640$. This closed-form labelling made it possible to generate a fully labelled, 22,315-image detection dataset at effectively zero annotation cost.



Figure 1: Copy-paste compositing pipeline applied to a real 100-Taka note: (a) background-free source note; (b) real COCO background; (c) the note after perspective warp, rotation, scaling, and colour jitter; (d) final composite with the automatically generated bounding box and class label.

E. Augmentation Operations

Table 2 lists the domain-randomisation operations applied per composite. Ranges were chosen to span, and in most cases slightly exceed, the variation expected from a hand-held webcam.

F. Final Dataset Composition

Four composites were produced per source note, plus a 10% share of background-only negative images with empty labels. The data was split 80/10/10 by *source note*, so that no individual note appears in more than one split, preventing information leakage (Table 3).

Table 2: Domain-randomisation augmentation operations

Stage	Operation	Setting
Geometry	Perspective warp	70% prob., jitter 8%
	Rotation	$\pm 30^\circ$
	Scale (frac. of frame)	0.25–0.90
	Placement	random
Photometric	HSV jitter	$H \pm 18^\circ$ $S \times 0.5\text{--}1.5$ $V \times 0.4\text{--}1.6$
	Brightness/contrast	0.4–1.6 / 0.7–1.3
Sensor	Motion blur	20% prob., kernel 3/5/7
	Gaussian noise	50% prob., $\sigma \approx 12$
	JPEG recompress.	quality 55–92
Balance	Negative images	10% of output

Table 3: Detection dataset composition after compositing

Split	Positive	Negative	Total	Instances
Train	16,224	1,620	17,844	16,224
Valid	2,016	217	2,233	2,016
Test	2,052	186	2,238	2,052
Total	20,292	2,023	22,315	20,292

IV. METHODOLOGY

A. Software Stack

The compositing engine uses Pillow and OpenCV for image transforms and NumPy for array mathematics. Training, validation, and export use the Ultralytics YOLOv8 framework (v8.4.64) on PyTorch 2.6.0 with CUDA 12.4. All experiments ran on an NVIDIA RTX 4060 Laptop GPU (8 GB) with an AMD Ryzen 7 CPU.

B. Detection Algorithm and Architecture

YOLOv8 is a single-stage, anchor-free detector: rather than matching objects to a fixed set of pre-defined anchor-box shapes, it predicts box edges directly from each grid cell, which simplifies training and improves accuracy on unusual aspect ratios, relevant here since banknotes are long and thin. We fine-tune the “s” (small) variant (11.1M parameters, 28.5 GFLOPs), which offers the best accuracy/speed trade-off for our hardware. The architecture follows the standard backbone/neck/head structure (Fig. 3): a CSPDarknet backbone built from C2f blocks and an SPPF module extracts multi-scale features; a PAN-FPN neck fuses features across depths so notes at very different scales are both handled well;

and a decoupled, anchor-free head predicts class probabilities and box coordinates at three spatial scales. Predictions are filtered by confidence and de-duplicated with Non-Maximum Suppression.

C. Loss Function

Training minimises a weighted sum of CIoU box-regression loss, Distribution Focal Loss (DFL) for sub-pixel localisation, and binary cross-entropy classification loss, with the default Ultralytics weightings (box 7.5, class 0.5, DFL 1.5).

D. Transfer Learning and Training Procedure

The model is initialised from COCO-pretrained weights rather than trained from scratch, which sped convergence substantially: mAP@0.5 exceeded 0.97 within eight epochs. Training ran for 80 epochs at 640×640 resolution, batch size 16, with the SGD optimiser (lr=0.01, momentum=0.937, weight decay 0.0005, 3-epoch warmup). Online mosaic (1.0) and mixup (0.15) augmentation were layered on top of the offline compositing described in Section III, while horizontal and vertical flips were disabled, since a note’s orientation carries information that mirroring would corrupt. Early-stopping patience was set to 15 epochs.

V. RESULTS AND ANALYSIS

A. Evaluation Metrics

Detectors are judged by precision, recall, and mean Average Precision (mAP). mAP@0.5 counts a detection as correct at $\text{IoU} \geq 0.5$; the stricter mAP@0.5:0.95 averages over IoU thresholds from 0.5 to 0.95 and is far more sensitive to precise localisation.

B. Overall Performance

Training converged rapidly: mAP@0.5 exceeded 0.97 by epoch 8 and 0.99 by epoch 11, after which mAP@0.5:0.95 continued to climb gradually as box localisation sharpened. Table 4 reports final test-set performance.

Table 4: Test-set performance (best checkpoint, 2,238 images)

Metric	Value
Precision	0.998
Recall	0.997
mAP@0.5	0.995
mAP@0.5:0.95	0.847
Inference speed	7.3 ms/img (≈ 135 FPS)
Preprocess / postprocess	1.7 ms / 1.7 ms

The gap between mAP@0.5 (0.995) and mAP@0.5:0.95 (0.847) is worth interpreting rather than only reporting: it indicates that the detector reliably finds and correctly classifies the great majority of notes (driving the high mAP@0.5), but

Figure 1 — Methodology Pipeline

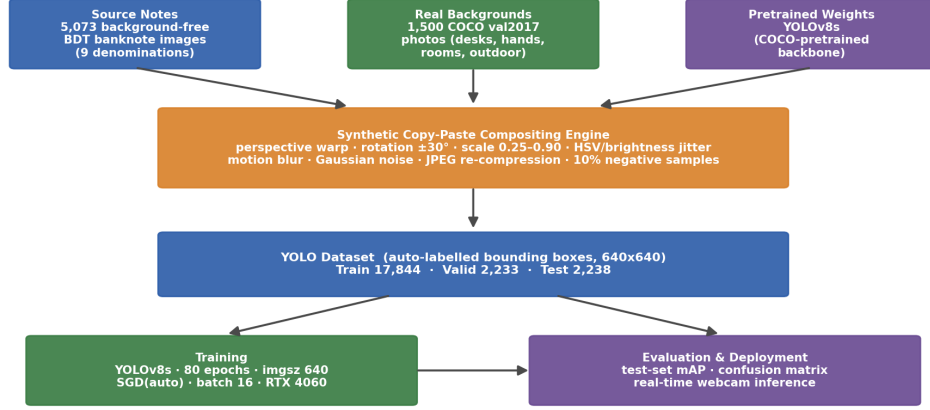


Figure 2: End-to-end methodology pipeline: source notes, real COCO backgrounds, and COCO-pretrained YOLOv8s weights feed the synthetic copy-paste compositing engine, which produces the labelled YOLO detection dataset used for training, evaluation, and deployment.

Figure 2 — YOLOv8s Detection Architecture

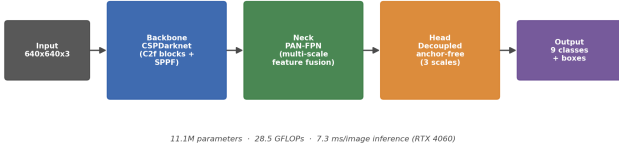


Figure 3: YOLOv8s detection architecture: input $\rightarrow$ CSP-Darknet backbone $\rightarrow$ PAN-FPN neck $\rightarrow$ decoupled anchor-free head $\rightarrow$ output (9 classes + box coordinates). 11.1M parameters, 28.5 GFLOPs, 7.3 ms/image on an RTX 4060.

that box localisation precision softens somewhat at the tighter IoU thresholds averaged into mAP@0.5:0.95. A plausible contributing factor is that pasted, warped note boundaries in the synthetic training and test data carry mild anti-aliasing that does not correspond to a pixel-perfect physical edge, a hypothesis we have not yet verified directly and flag as worth investigating alongside the real-world validation described in Section VIII.

C. Per-Class Results

Table 5 shows performance broken down by denomination. Results are uniformly strong across all nine classes, including the least-represented source class (10 Taka, 419 source images), confirming the model does not simply favour the most frequent class (500 Taka, 1,243 source images). We note that mAP@0.5 rounds to an identical 0.995 across every individual class; mAP@0.5:0.95, which is more sensitive to localisation and shows genuine per-class spread (0.838–0.858), is

the more informative column for distinguishing class-level performance and should be preferred over mAP@0.5 when comparing denominations.

Table 5: Per-class detection metrics on the test set

Class	P	R	mAP@.5	mAP@.5:95
2 Taka	0.998	1.000	0.995	0.838
5 Taka	0.998	0.996	0.995	0.844
10 Taka	0.995	0.988	0.995	0.848
20 Taka	0.998	1.000	0.995	0.847
50 Taka	1.000	0.993	0.995	0.851
100 Taka	0.999	1.000	0.995	0.845
200 Taka	0.999	1.000	0.995	0.843
500 Taka	1.000	1.000	0.995	0.853
1000 Taka	0.994	1.000	0.995	0.858
All	0.998	0.997	0.995	0.847

D. Confusion Matrix

The normalised confusion matrix (Fig. 5) is close to perfectly diagonal, with predicted-to-true confidence of 0.99–1.00 on every denomination. The only residual error is occasional confusion with the background class, a missed or spurious detection, never a mix-up between two different denominations. For a currency-reading application this is the safer of the two possible failure modes: the system practically never reports the wrong denomination, only, rarely, that it failed to detect a note at all.

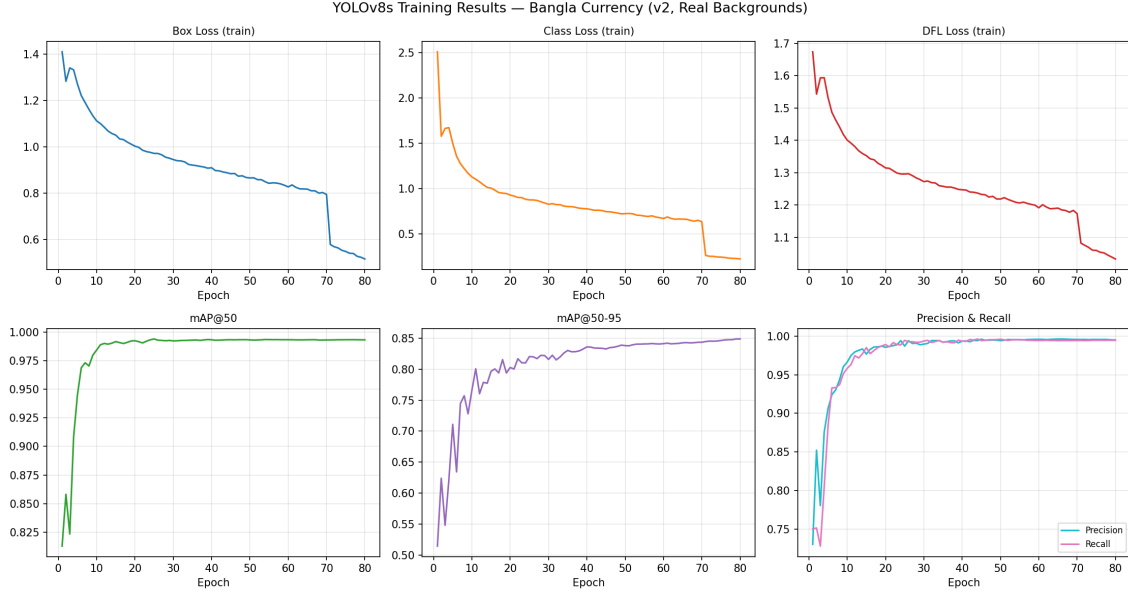


Figure 4: Training curves: box/class/DFL losses (top row) and mAP@50, mAP@50-95, precision/recall (bottom row) versus epoch. mAP@0.5 exceeds 0.97 by epoch 8 and 0.99 by epoch 11; mAP@0.5:0.95 continues climbing gradually through epoch 80 as box localisation sharpens.

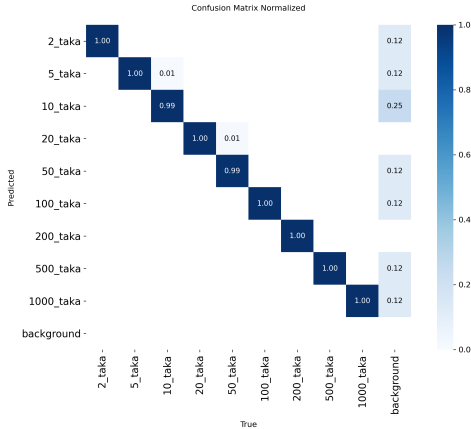


Figure 5: Normalised confusion matrix on the test set. The diagonal is 0.99–1.00 for every denomination; residual error is limited to the background column (missed/spurious detections), never cross-denomination confusion.

E. Qualitative Detections

Fig. 6 shows example predictions on a validation batch, with predicted boxes, labels, and confidence scores overlaid. Notes are tightly localised across a range of scales, rotations, and composited backgrounds, exactly the conditions the compositing pipeline (Section III) was designed to reproduce. We note explicitly that these qualitative examples, like the quantitative results above, are drawn from held-out synthetic composites rather than independently photographed real notes; see Section VIII.

VI. COMPARISON WITH RELATED WORK

Table 6 situates our results among comparable Bangladeshi and multi-currency detection and recognition systems. Direct numerical comparison should be read with care, since each study uses a different dataset, class set, and test protocol; the table shows our system is competitive with, or exceeds, published detection baselines while targeting real-world robustness through training-data design.

Table 6: Comparison with related currency-recognition systems

Work	Task	mAP.5	.5:.95	P/R (acc)
Axis-sym. masks [1]	Classify	—	—	98.6% acc
CNN real-time [2]	Classify	—	—	high acc
Dual-stream [3]	Classify	—	—	robust
YOLOv5 Taka [4]	Detect	—	—	0.95/0.93
YOLOv8n-SE [5]	Detect	0.972	0.804	0.965/0.952
This work	Detect	0.995	0.847	0.998/0.997

Our mAP@0.5:0.95 of 0.847, the metric most sensitive to localisation quality, exceeds the 0.804 reported by the custom-head YOLOv8n multi-currency detector [5], despite our use of an unmodified backbone; we attribute this gain to the domain-randomised training data rather than architectural modification. Our precision and recall (0.998/0.997) ex-

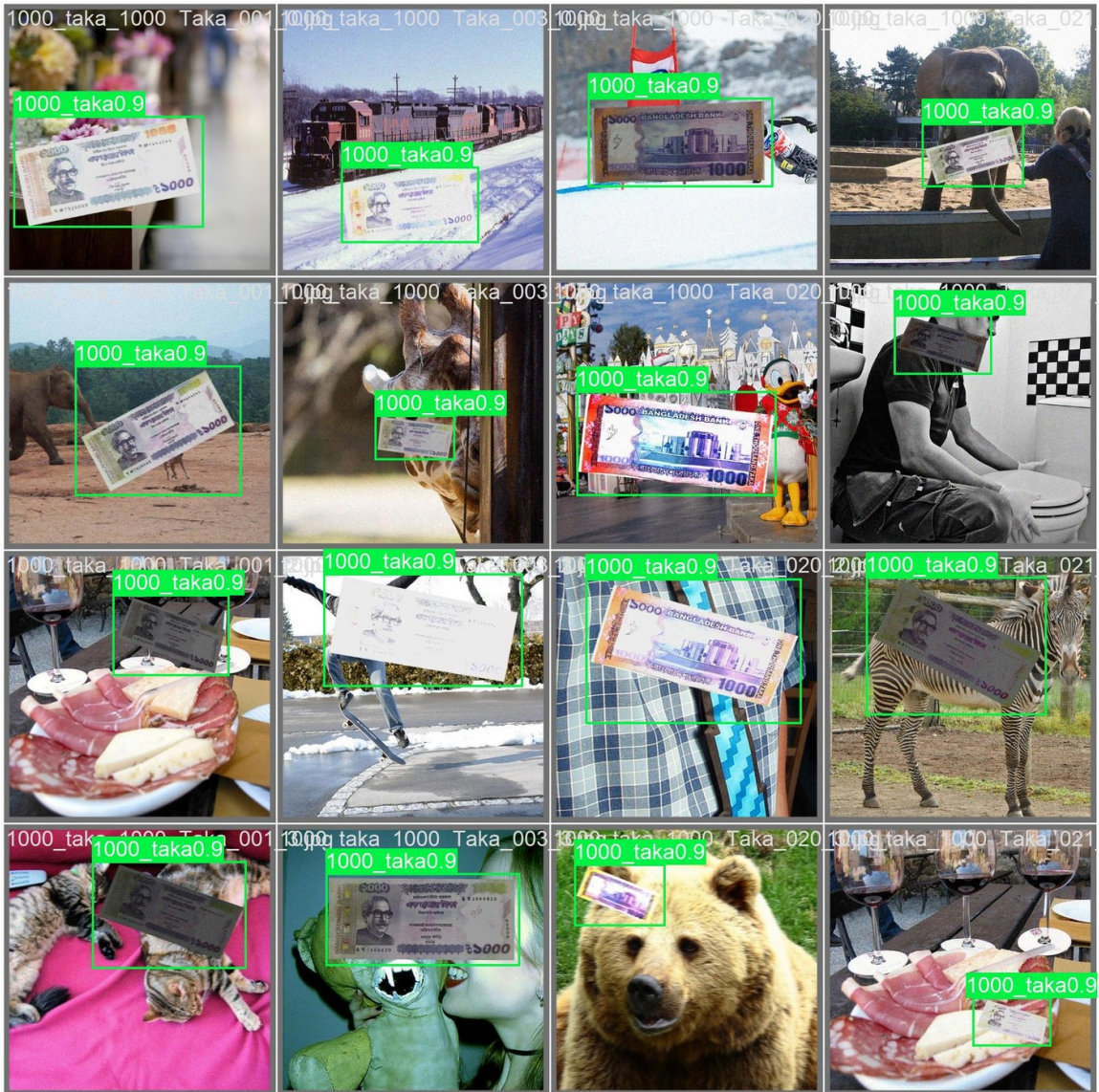


Figure 6: Example predictions on a validation batch (predicted label + confidence score). The detector correctly localises the 1000-Taka note across widely varying scales, rotations, and cluttered composited backgrounds.

ceed the YOLOv5 baseline’s 0.95/0.93 [4], and our labelled dataset (22,315 images) is roughly seven times larger than that baseline’s 3,111 hand-annotated images, obtained at zero manual annotation cost. Unlike the classification-only systems [1, 2, 3], our model outputs bounding boxes and therefore supports localisation and multi-note counting, not only single-note recognition.

VII. REAL-TIME DEPLOYMENT

The trained model is deployed in a webcam application built on OpenCV, annotating live frames with bounding boxes at interactive frame rates. An automatic brightness-correction step applies gain-and-gamma correction when a frame’s mean intensity falls below a threshold, addressing a practical low-light failure mode observed during testing. The model

is additionally exported to TorchScript for dependency-free inference in production environments.

VIII. LIMITATIONS AND THREATS TO VALIDITY

We report the following limitations directly, as we consider them integral to interpreting the results above rather than incidental caveats.

Evaluation is synthetic-only. Every quantitative result in Section V, and every qualitative example, is measured on held-out synthetic composites generated by the same pipeline used for training, not on independently captured photographs of real physical notes in real environments. This is the paper’s most significant limitation: while the compositing pipeline was explicitly designed to resemble webcam deploy-

ment conditions (Section III), and the system has been informally exercised live during development, no held-out real-world test set with independently verified ground truth has yet been evaluated, and the reported metrics should be read as an estimate of performance on data statistically similar to training data, not as a validated real-world accuracy figure.

Condition and authenticity. The system verifies denomination, not authenticity, and source notes are relatively good-condition; worn, torn, or heavily folded notes and counterfeit detection remain open problems.

Synthetic realism. Composited notes lack real cast shadows, physical creases, and true occlusion by other objects, all present in genuine handheld photographs.

IX. CONCLUSION AND FUTURE WORK

We have presented a reproducible pipeline for real-time Bangladeshi Taka detection that converts a clean classification dataset into a large, domain-randomised detection dataset via automatic copy-paste compositing, and fine-tunes a YOLOv8s detector on it. The resulting model achieves 0.995 mAP@0.5 and 0.847 mAP@0.5:0.95 on a held-out synthetic test set, with near-perfect precision and recall, and runs above 100 FPS on a laptop GPU. The central methodological lesson is that engineering training data to match deployment conditions, real backgrounds plus aggressive, physically motivated augmentation, is what turns a brittle studio-only classifier into a detector designed for real-world robustness.

The most important next step, ahead of any architectural refinement, is closing the validation gap named in Section VIII: collecting and evaluating against an independently captured, hand-photographed real-world test set. Further future work includes INT8 quantisation and ONNX/TensorRT/TFLite export for embedded deployment, counterfeit and condition detection, and integration into a wearable assistive-reading platform with audio feedback for visually impaired users.

REFERENCES

- [1] “Bangladeshi Banknote Recognition by Neural Network with Axis-Symmetrical Masks,” reporting 98.57% average accuracy over eight denominations.
- [2] “Bangladeshi Banknote Recognition in Real-Time Using Convolutional Neural Network for Visually Impaired People.”
- [3] “Robust and Real-Time Bangladeshi Currency Recognition: A Dual-Stream MobileNet and EfficientNet Approach,” arXiv:2602.07015.
- [4] “NSTU-BDTAKA: An Open Dataset for Bangladeshi Paper Currency Detection and Recognition,” 2024; YOLOv5-based detection subset (3,111 annotated images), precision 0.95, recall 0.93.
- [5] “Real-Time Currency Detection and Voice Feedback for Visually Impaired Individuals,” arXiv:2510.20267 (custom-head YOLOv8n, USD/EUR/BDT).
- [6] Nino, Gardi, et al., “Synthetic-to-Real Object Detection Using YOLO and Domain Randomization Strategies,” arXiv:2509.15045.
- [7] “A Diverse Image Dataset for Bangladeshi Currency Recognition” (BanglaTaka), Mendeley Data, 5,073 images, 9 denominations. <https://data.mendeley.com/datasets/3cv2syppkkh/1>
- [8] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in *Proc. ECCV*, 2014.
- [9] G. Jocher et al., “Ultralytics YOLOv8” (v8.4.64). <https://github.com/ultralytics/ultralytics>